

BÁO CÁO BÀI THỰC HÀNH SỐ 3

Dựng Agent đặt vé máy bay bằng LangChain, LangGraph: Lớp harness và 3 mẫu thiết kế Agent

Họ và tên: Phan Việt Hoàng
Mã số sinh viên: 24520565
Môn học: SE373

1. Tổng quan

Bài toán: đặt 1 vé SGN → DAD ngày 2026-10-07, khởi hành trước 12:00, giá tối đa 2.000.000 VND. Công cụ (tool) là dữ liệu giả (mockup), còn model là một LLM thật gọi qua API. Bài làm theo công thức Agent = Model + Harness: model quyết định làm gì tiếp theo; harness (code Python thường) quyết định việc đó có được phép không và công việc đã xong thật chưa.

1.1. LangChain và LangGraph

- LangChain:** cung cấp `create_agent` (vòng lặp ReAct), middleware (`wrap_tool_call` để chặn mọi tool call, `ModelCallLimitMiddleware` để giới hạn số lần gọi model), prompt template và structured output (Pydantic) để model trả về kế hoạch có cấu trúc.
- LangGraph:** mô tả agent như một đồ thị trạng thái (StateGraph): mỗi bước là một node, việc rẽ nhánh là conditional edge. `create_agent` của LangChain thực chất chạy trên LangGraph. Mình dùng LangGraph trực tiếp cho mẫu Hybrid vì luồng có nhánh rõ ràng (plan → review → execute → fallback).

1.2. Cấu trúc mã nguồn

File	Vai trò
harness.py	Harness dùng chung, tool mockup, 4 kịch bản, tạo LLM thật từ cấu hình .env, đếm token
agent_react.py	Mẫu 1: ReAct (LangChain <code>create_agent</code>)
agent_plan_execute.py	Mẫu 2: Plan-then-Execute (prompt template + structured output + người duyệt)
agent_hybrid.py	Mẫu 3: Hybrid = Plan-then-Execute + ReAct fallback (LangGraph)
evaluate.py	Chạy 3 mẫu × 4 kịch bản × N lần, in bảng, ghi <code>evaluation_results.md</code> / <code>.json</code>
requirements.txt, .env.example	Thư viện cần cài và mẫu file khai báo API key

2. Yêu cầu 1: các lớp harness

Cả 3 mẫu dùng chung một harness (`harness.py`) để so sánh công bằng: mọi tool call, dù đến từ ReAct hay từ code thực thi kế hoạch, đều đi qua một cổng duy nhất là `guarded_call()`.

Lớp	Hàm / lớp	Cách hoạt động	Lỗi nó chặn
1. Ràng buộc là dữ liệu	Constraints	Yêu cầu của người dùng lưu thành dataclass. Prompt sinh ra từ nó, và <code>is_ok()</code> kiểm tra bằng code.	Model quên ràng buộc (goal drift)
2. Kiểm quyền	<code>check_permission()</code>	Chạy TRƯỚC tool. <code>book_seat</code> chỉ cho chuyển thỏa ràng buộc. <code>pay</code> chỉ cho booking thỏa ràng buộc và chưa	Đặt/ trả tiền sai chuyển

Lớp	Hàm / lớp	Cách hoạt động	Lỗi nó chặn
		trả tiền (tiền không hoàn lại được nên kiểm chặt nhất).	
3. Hoàn thành kiểm bằng code	is_done()	Đọc lại booking từ hệ thống: có booking, đã trả tiền, thỏa ràng buộc mới là DONE. Không tin câu "xong rồi" của model.	Model báo xong nhưng chưa xong
4. Bàn giao	handoff()	Khi FAILED, trả cho người 3 thứ: đã làm gì, đã thử gì, và 1 câu hỏi cụ thể (nói giờ/giá, hay thử lại).	Agent kẹt mà không ai biết
+ Ngân sách	MAX_TOOL_CALLS, ModelCallLimitMiddleware	Giới hạn 12 tool call và số lần gọi model mỗi lần chạy.	Vòng lặp vô hạn, tốn chi phí

3. Yêu cầu 2: ba mẫu thiết kế agent

3.1. ReAct

Model được gọi lại sau mỗi kết quả tool và quyết định MỘT bước tiếp theo (search → book → pay). Ưu điểm: thích nghi tốt, thấy lỗi thì đổi cách (thử lại, chọn chuyển khác). Nhược điểm: nhiều lần gọi model, người không thấy trước kế hoạch.

3.2. Plan-then-Execute

Model được gọi 1 lần để viết cả kế hoạch dạng Plan(steps). Người duyệt kế hoạch (approve / reject; bị từ chối thì model viết lại, tối đa 2 plan). Nếu duyệt, code thường chạy tuần tự từng bước mà KHÔNG gọi model nữa. Mã booking chưa biết lúc lập kế hoạch nên model viết "\$booking_code" và code điền vào sau. Ưu điểm: rõ, nhìn thấy kế hoạch trước khi tốn tiền. Nhược điểm: nếu một bước lỗi thì kế hoạch không tự đổi được, chỉ có thể dừng và bàn giao.

3.3. Hybrid (Plan-then-Execute + ReAct fallback, dùng LangGraph)

Đi đường rõ và dễ duyệt khi mọi thứ suôn sẻ; chỉ khi một bước lỗi mới giao phần còn lại cho một agent ReAct để thích nghi. Đồ thị:

plan → review: model viết kế hoạch, người (hoặc reviewer tự động) duyệt.
review → execute nếu được duyệt; quay lại plan nếu bị từ chối và còn lượt; kết thúc nếu kế hoạch rỗng (không có chuyến hợp lệ) hoặc hết lượt.
execute → kết thúc nếu mọi bước OK; sang react nếu có bước lỗi.
react: agent ReAct nhận bối cảnh (bước nào lỗi, đã có booking nào, danh sách chuyến) và hoàn tất theo cách khác. Vẫn đi qua cùng harness.

Kết quả cuối cùng của cả 3 mẫu đều do harness quyết định (is_done / handoff), không phải do model.

4. Yêu cầu 3: đánh giá

4.1. Kịch bản và chỉ số

Kịch bản	Mô tả	Kết quả đúng
success	Có chuyến hợp lệ (VN122), không có sự cố	DONE
no_valid	Không chuyến nào vừa sáng vừa ≤ 2 triệu; model "quên" giá và định đặt VJ604	FAILED + handoff

Kịch bản	Mô tả	Kết quả đúng
sold_out	VN122 hết chỗ nhưng còn chuyển dự phòng hợp lệ VU330	DONE
transient	book_seat báo timeout 1 lần, thử lại thì được	DONE

Chỉ số: kết quả có đúng kỳ vọng không; số lần gọi model (chi phí); số tool call; số lần harness chặn; số tiền trả sai (unsafe payments, phải bằng 0).

Phương pháp: model là LLM thật gọi qua API tương thích OpenAI (API key, tên model và URL khai báo trong file .env: LLM_API_KEY, LLM_MODEL, LLM_BASE_URL), temperature = 0. Tool vẫn là mockup. Vì LLM thật không lặp lại hoàn toàn, mỗi cặp (mẫu, kịch bản) được chạy N lần (N = 3) rồi lấy tỉ lệ đúng và giá trị trung bình. "Người duyệt kế hoạch" trong Plan-then-Execute và Hybrid là một hàm kiểm tra bằng code (từ chối kế hoạch đặt chuyển vi phạm ràng buộc), không phải LLM.

Chi phí được đo bằng số lần gọi model và tổng token (callback đếm trực tiếp trên model).

4.2. Kết quả

Kết quả của python evaluate.py với N = 3: 3 mẫu × 4 kịch bản × 3 lần = 36 lượt chạy, không có lượt nào ra ERROR. Cột "Đúng" là số lần chạy ra kết quả đúng kỳ vọng trên tổng số lần chạy; các cột còn lại là giá trị trung bình trên 3 lần chạy. Số tiền trả sai bằng 0 ở cả 36 lượt (xem bảng tổng hợp).

Mẫu	Kịch bản	Đúng	TB gọi model	TB token	TB tool call	Bị chặn
ReAct	success	3/3	4.0	6009	3.0	0
ReAct	no_valid	3/3	2.0	2905	1.0	0
ReAct	sold_out	3/3	5.3	8527	4.3	0
ReAct	transient	3/3	5.3	8292	4.3	0
Plan-then-Execute	success	3/3	1.0	1598	4.0	0
Plan-then-Execute	no_valid	3/3	1.0	1331	1.0	0
Plan-then-Execute	sold_out	0/3	1.0	1898	2.0	0
Plan-then-Execute	transient	0/3	1.0	1544	2.0	0
Hybrid	success	3/3	1.0	1454	4.0	0
Hybrid	no_valid	3/3	1.0	1279	1.0	0
Hybrid	sold_out	3/3	4.0	6840	4.0	0
Hybrid	transient	3/3	4.0	6430	4.0	0

Mẫu	Đúng	TB gọi model	TB token	Trả tiền sai
ReAct	12/12	4.17	6433	0
Plan-then-Execute	6/12	1.00	1593	0
Hybrid	12/12	2.50	4001	0

4.3. Phân tích

Độ đúng. ReAct và Hybrid đúng 12/12 lượt, ở cả 4 kịch bản. Plan-then-Execute chỉ đúng 6/12: đúng 3/3 ở success và no_valid nhưng sai 0/3 ở sold_out và transient. Ở sold_out, book_seat(VN122) trả về sold_out; ở transient, book_seat báo timeout. Kế

hoạch đã viết trước không tự đổi được, nên code dừng sau 2 tool call (search và book) rồi bàn giao, kết quả là FAILED trong khi kỳ vọng là DONE.

Chi phí khi suôn sẻ. Ở success, Plan-then-Execute và Hybrid chỉ gọi model 1 lần, khoảng 1.5 nghìn token (1598 và 1454). ReAct gọi 4 lần và dùng 6009 token, gấp khoảng 4 lần, vì phải gọi lại model sau mỗi kết quả tool. Ở no_valid cũng tương tự (1279 và 1331 token so với 2905 của ReAct). Chênh lệch nhỏ giữa Hybrid (1454) và Plan-then-Execute (1598) ở success chỉ là do độ dài câu trả lời của model; hai mẫu chạy cùng một đường đi nên không coi là Hybrid rẻ hơn thật sự.

Chi phí khi có sự cố. Ở sold_out và transient, Hybrid tốn 4.0 lần gọi model và 6840 / 6430 token, vì ngoài 1 lần lập kế hoạch còn phải chạy ReAct fallback. Con số này vẫn thấp hơn ReAct (5.3 lần gọi, 8527 / 8292 token), tức ít hơn khoảng 20 đến 22% token. Tính chung 12 lượt, Hybrid dùng trung bình 4001 token so với 6433 của ReAct (ít hơn khoảng 38%) mà vẫn đúng 12/12. Plan-then-Execute rẻ nhất (1593 token) nhưng cái giá là sai ở hai kịch bản có sự cố; nó rẻ một phần vì dừng sớm chứ không phải vì làm xong việc.

Harness. Trả tiền sai bằng 0 ở mọi lượt, đúng như yêu cầu. Tuy nhiên cột "Bị chặn" cũng bằng 0 ở mọi ô: trong lần chạy này model chưa lần nào đề xuất đặt hay trả tiền cho chuyển vi phạm ràng buộc (ở no_valid, ReAct thấy không có chuyển hợp lệ thì dừng, còn hai mẫu có kế hoạch thì viết kế hoạch rỗng). Vì vậy kết quả này cho thấy model làm đúng, nhưng chưa chứng minh bằng số liệu rằng check_permission chặn được hành vi sai.

Độ ổn định. Không có lượt nào ra ERROR. Mỗi ô (mẫu, kịch bản) hoặc đúng 3/3 hoặc sai 0/3, không có ô nào lúc đúng lúc sai, có thể vì temperature = 0 và kịch bản mockup khá đơn giản. Số lần gọi model và tool call lẻ ở ReAct (5.3 và 4.3 ở sold_out, transient) cho thấy đường đi của model vẫn dao động nhẹ giữa các lần chạy.

Đối chiếu với dự kiến ban đầu: kết quả khớp với giả thuyết. ReAct thích nghi tốt (12/12) nhưng tốn nhiều lần gọi model nhất (4.17 lần). Plan-then-Execute rẻ nhất (1.00 lần) nhưng không xử lý được sự cố giữa chừng (6/12). Hybrid giữ chi phí thấp khi suôn sẻ và thích nghi được khi có lỗi (12/12, 2.50 lần gọi).

5. Kết luận và hạn chế

Tình huống	Mẫu phù hợp (theo thiết kế)
Việc quen thuộc, ít sự cố, cần người duyệt trước khi tốn tiền	Plan-then-Execute
Môi trường nhiều bất định, phải khám phá từng bước	ReAct
Phần lớn chạy suôn sẻ nhưng thỉnh thoảng có sự cố cần xử lý	Hybrid

Số liệu khớp với bảng thiết kế ở trên. Trong 4 kịch bản, Hybrid cho đánh đổi tốt nhất: đúng 12/12 như ReAct nhưng dùng ít token hơn khoảng 38% (4001 so với 6433). ReAct đúng hết nhưng đắt nhất khi mọi thứ suôn sẻ. Plan-then-Execute rẻ nhất nhưng chỉ đúng 6/12 vì không thích nghi được khi book_seat gặp lỗi. Nhận định này chỉ dựa trên 4 kịch bản mockup, mỗi kịch bản 3 lần chạy.

Điểm chắc chắn theo thiết kế: harness mới là thứ tạo an toàn (ràng buộc là dữ liệu, kiểm quyền trước khi chạy, hoàn thành do code kiểm, bàn giao khi thất bại), còn mẫu thiết kế chỉ quyết định đánh đổi giữa chi phí và khả năng thích nghi.

Hạn chế: (1) chỉ có 4 kịch bản và N lần chạy mỗi kịch bản, số liệu mang tính minh họa chứ chưa đủ để kết luận thống kê; (2) kết quả phụ thuộc vào LLM được chọn; (3) tool là mockup, chưa có độ trễ, lỗi mạng thật; (4) người duyệt kế hoạch là hàm tự động; (5) số lần harness chặn bằng 0 nên lớp kiểm quyền chưa được kích hoạt thật trong lần chạy này. Hướng mở

rộng: thêm kịch bản ép model đề xuất chuyển vi phạm ràng buộc để kiểm tra `check_permission`, thêm kịch bản lỗi (kết quả rỗng, vòng lặp, bịa dữ liệu như trong `flight_agent_failure_mode.py`), thử nhiều LLM, tăng N.

6. Cách chạy

- `pip install -r requirements.txt` (dùng langchain-openai, gọi được mọi nhà cung cấp có API tương thích OpenAI qua `LLM_BASE_URL`)
- Sao chép `.env.example` thành `.env` và điền 3 giá trị: `LLM_API_KEY` (API key), `LLM_MODEL` (tên model, ví dụ `gpt-4o-mini`) và `LLM_BASE_URL` (URL API, ví dụ `https://api.openai.com/v1`).
- `python evaluate.py --runs 3` chạy 3 mẫu × 4 kịch bản × 3 lần, ghi `evaluation_results.md` và `.json`
- `python agent_react.py`, `python agent_plan_execute.py`, `python agent_hybrid.py`
demo từng mẫu (chọn kịch bản 1 đến 4; hai mẫu sau hỏi y/n để duyệt kế hoạch)